

Vivriti Co-Lending Control Tower: System Architecture & Scale Design

Version: 1.0.0 | **Author:** Co-Lending Control Tower Engineering | **Classification:** Technical Architecture Document

1. Executive Summary & Problem Space

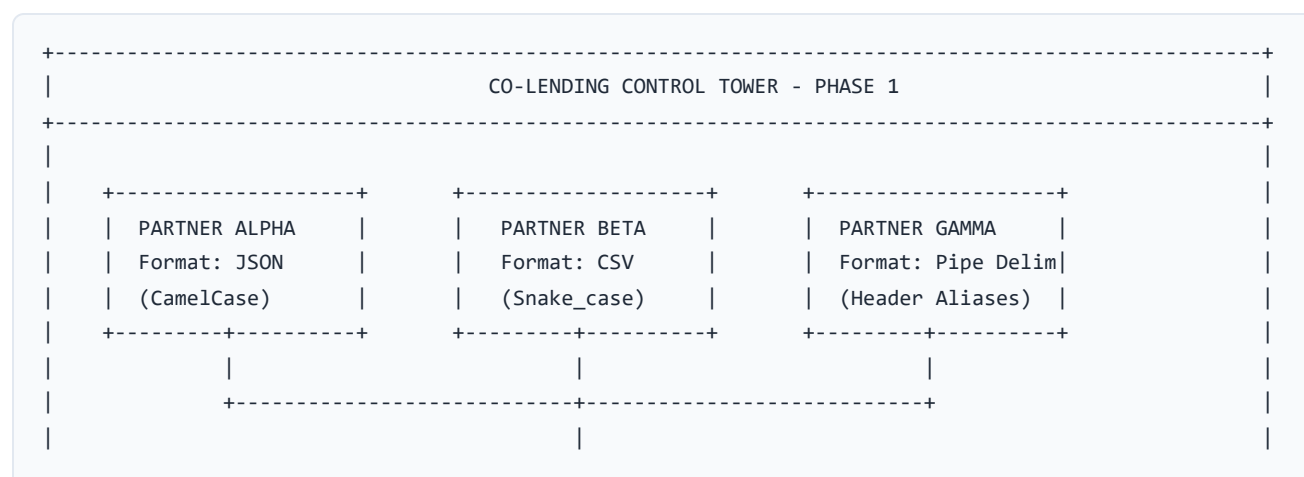
Co-lending in modern Indian digital credit infrastructure involves tri-party capital orchestration among **Originator (LSP/Fintech)**, **Funding Bank**, and **LMS (Core Banking/Loan Management System)**. The primary operational challenges are:

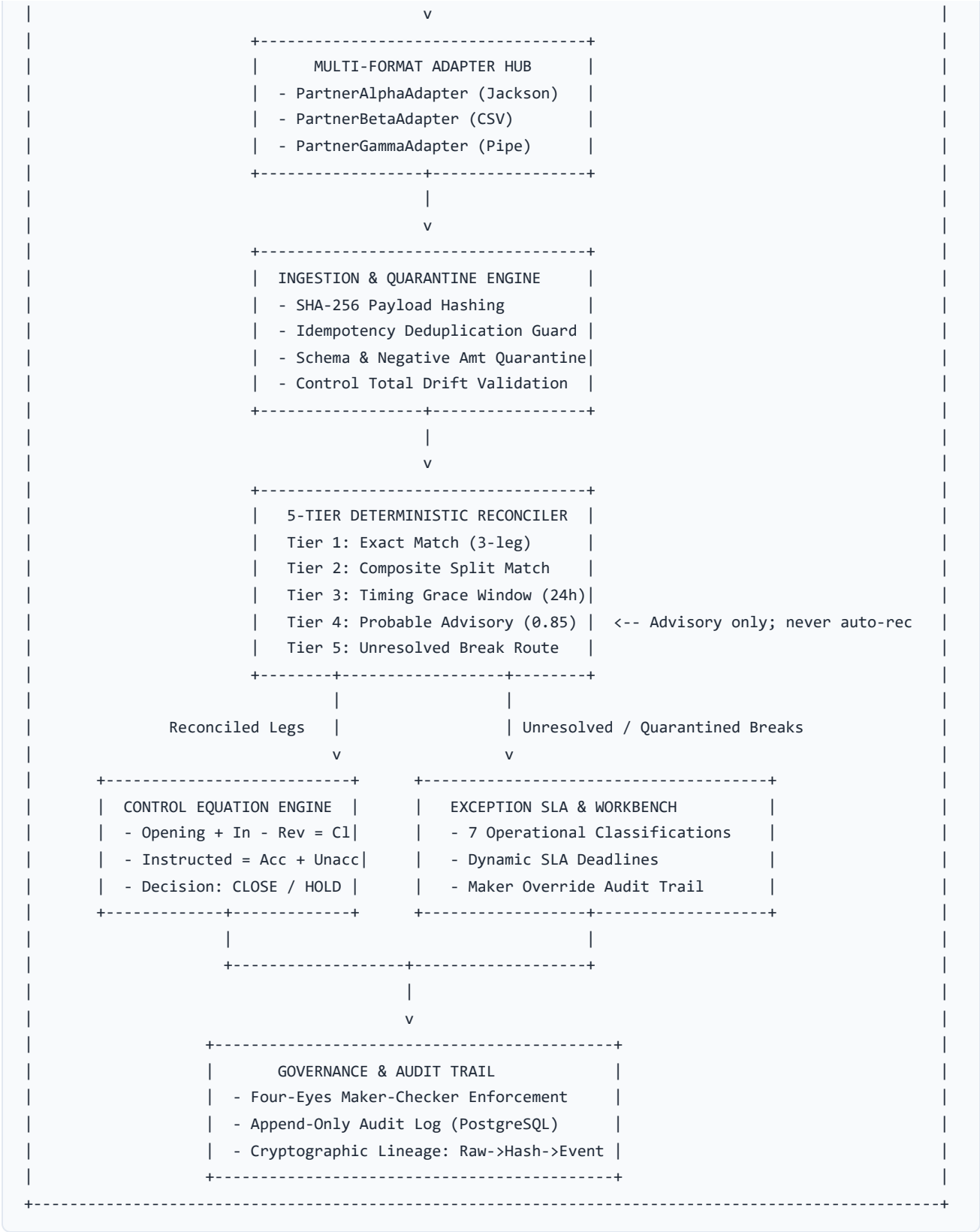
- **Asynchronous Feed Ingestion:** Disparate arrival times, formats (JSON, CSV, Pipe-delimited text), and schema versions across partners.
- **Timing & Cutoff Dynamics:** Settlement debits cleared post-cutoff (e.g. 18:00 UTC) must be accommodated without prematurely declaring accounting breaks.
- **Zero False-Match Mandate:** A false positive match in disbursements (e.g., matching a failed bank debit to an active loan booking) creates unrecoverable financial exposure.
- **Idempotency & Replay Protection:** Webhook retries and multi-day callback replays must never produce duplicate financial postings.
- **Maker-Checker Governance:** Complete separation of duties prevents single-actor override and balance-sheet closure.

This document outlines the **Phase 1 Deterministic Architecture** (production core built in Java 21 / Spring Boot / PostgreSQL) and the **Phase 2 Streaming Scale Architecture** designed for 10,000 TPS and 50M daily transactions.

2. Phase 1 Architecture: Deterministic Financial Core

2.1 High-Level Architectural Topology





2.2 Ingestion & Failure Isolation Mechanics

- **Raw-to-Canonical Normalization:** Every ingested record is persisted into `raw_source_record` with its calculated SHA-256 payload hash before parsing.
- **Idempotency Guarantee:** A composite unique constraint on `(batch_id, payload_hash)` ensures that duplicate webhooks or re-ingested feeds are safely skipped without side effects.

- **Quarantine Isolation:** Malformed records (missing loan references, negative amounts without reversal markers, corrupt dates) are marked `quarantined=true` with detailed error codes. Valid records in the same batch continue processing without interruption.
- **Batch Drift Validation:** The declared count and paise totals in the batch header are verified against calculated sums. If a discrepancy exists, the batch is flagged with `BATCH_TOTAL_DRIFT`.

2.3 5-Tier Reconciliation Hierarchy

Tier	Match Strategy	Invariant Checked	Confidence	Operational Result
1	Exact Match	All 3 legs present (<code>ORIGINATOR</code> , <code>BANK</code> , <code>LMS</code>), identical paise, identical loan reference, status <code>SUCCESS</code>	1.0000	Reconciled straight-through (STP)
2	Composite Split	1 Originator instruction matched to $\$N\$$ Bank settlement debits where $\$ \sum \text{Bank Paise} = \text{Originator Paise} \$$, LMS matches total	0.9500	Multi-leg balanced reconciliation
3	Timing Difference	Originator and LMS booked, Bank debit cleared post-cutoff (18:00 UTC) but within 24h grace window	0.9000	Classified as <code>PENDING_GRACE_PERIOD</code> ; not escalated
4	Scored Probable	Proximity score $\$S = 0.4 \times \text{Amt} + 0.3 \times \text{Date} + 0.3 \times \text{Levenshtein}(\text{Ref}) \geq 0.85 \$$	$\$0.85 - 0.94 \$$	Advisory Only. Never auto-reconciled. Human review mandatory.
5	Unresolved Handler	Missing legs, amount mismatches, status conflicts, or non-matching records	0.0000	Created in <code>reconciliation_exception</code> with SLA and routed

2.4 Control Equations & Zero-Tolerance Balance Policy

Every batch must satisfy two fundamental financial control equations before closing:

$$\text{Equation 1: } \text{Opening Position} + \text{Valid Movements} - \text{Reversals} = \text{Closing Position}$$

$$\text{Equation 2: } \text{Total Instructed Paise} = \text{Matched} + \text{Timing Pending} + \text{Unresolved Exceptions} + \text{Quarantined} + \text{Unaccounted}$$

Close Policy:

- **CLOSE:** Granted only when $\text{Unaccounted Paise} = 0 \$$, $\text{Quarantined Paise} = 0 \$$, $\text{Unresolved Exception Paise} = 0 \$$, and batch status is `VALIDATED`.
- **HOLD:** Triggered if even **1 paise** remains unaccounted, or if open exceptions exist, or if the batch was quarantined due to total drift or schema violations. Silent write-offs or balancing journal entries are strictly blocked.

2.5 Four-Eyes Governance (Maker-Checker Guard)

- **Segregation of Duties:** `MakerCheckerGuard` inspects the immutable `audit_log` . If the actor attempting to approve a batch close (`ROLE_APPROVER`) has executed an override on any exception within that batch, the approval is rejected with HTTP 403 `SEGREGATION_VIOLATION` .

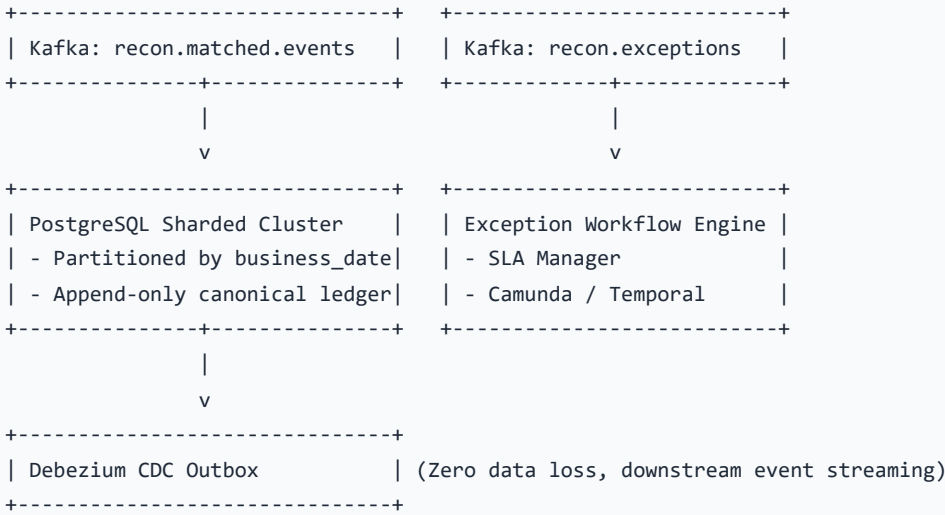
3. Phase 2 Scale Design: 10,000 TPS Distributed Architecture

3.1 Target Workload & Capacity Requirements

- **Peak Throughput:** 10,000 Transactions Per Second (TPS).
- **Daily Volume:** 50,000,000 disbursement and settlement events per day.
- **Latency Budgets:** Ingestion p99 $< 50\text{ ms}$, Recon p99 $< 500\text{ ms}$, End-to-End SLA $< 2.0\text{ s}$.
- **State Retention:** 24-hour tumbling and sliding window for grace periods and multi-leg callbacks.

3.2 Distributed Streaming Topology





3.3 Key Architectural Components

A. Partitioning & Co-Location Strategy

- **Partition Key:** `hash(partner_code, loan_reference) % 64` .
- **Guarantee:** All three legs (`ORIGINATOR` , `BANK` , `LMS`) for any loan reference are guaranteed to arrive on the **same Kafka partition** and be processed by the **same Flink stateful operator instance**. This eliminates distributed cross-node coordination during matching.

B. Multi-Layer Deduplication Engine

- **Layer 1 (Redis Bloom Filter):** High-speed probabilistic check ($p < 0.001$, memory footprint $\approx 120\text{MB}$) for 50M keys). Instantly filters 99.9% of seen payloads.
- **Layer 2 (Redis SETNX with 48h TTL):** Distributed lock and exact existence check on `sha256(payload)` .
- **Layer 3 (Database Constraint):** PostgreSQL primary key unique constraint on `(batch_id, payload_hash)` as the final ACID safety net.

C. Stateful Windowing & RocksDB State Management

- **State Backend:** Embedded RocksDB on NVMe SSD with asynchronous incremental checkpointing to S3/GCS every 60 seconds.
- **State Lifecycle:** When an event arrives, it is appended to the `LoanState` object for that key. If all 3 legs match, the matched event is emitted immediately, and state is purged.
- **Watermark & Grace Period:** Watermarks allow up to 15 minutes of out-of-order message arrival. At $T_{\text{cutoff}} + 24\text{ hours}$, any incomplete loan triggers a timer that emits an `UNRESOLVED` break to the exception topic.

D. Change Data Capture (CDC) Outbox Pattern

- The transactional engine writes match decisions, exceptions, and audit logs to local PostgreSQL partitions.
- **Debezium CDC** tails the PostgreSQL Write-Ahead Log (WAL) to publish events to downstream Kafka topics (`audit.events` , `recon.decisions`), ensuring zero dual-write inconsistencies and 100% durability.

3.4 Capacity Planning & Sizing Analysis (50M Events/Day)

- **Average Event Payload:** 512 bytes raw, 1.2 KB canonical representation.
- **Ingestion Ingress Bandwidth:**

$$\text{Throughput} = 10,000 \text{ events/sec} \times 1.2 \text{ KB} = 12.0 \text{ MB/sec} = 96 \text{ Mbps}$$

- **Daily Storage Requirement:**

$$\text{Storage} = 50,000,000 \times 2.5 \text{ KB (Raw + Canonical + Audit)} \approx 125 \text{ GB/day}$$

$$\text{30-Day Working Retention} = 3.75 \text{ TB}$$

- **Flink RocksDB Memory Footprint:**

$$50,000,000 \text{ active keys} \times 350 \text{ bytes per state} \approx 17.5 \text{ GB RAM across cluster}$$

Easily hosted across 8 worker nodes (each 16 vCPU, 64 GB RAM).

4. Phase 1 vs Phase 2 Comparison Matrix

Dimension	Phase 1 (Deterministic Core)	Phase 2 (Distributed Scale)
Primary Workload	Daily batch & micro-batch settlement	Real-time continuous streaming
Throughput Capacity	1,000 – 2,000 events/sec	10,000+ events/sec
Processing Engine	Spring Boot 3.3.5 / Java 21 / Virtual Threads	Apache Flink 1.18 + RocksDB
State Storage	PostgreSQL / H2 ACID tables	Partitioned In-Memory + RocksDB + Object Store
Deduplication	PostgreSQL unique constraints + SHA-256	Redis Bloom Filter + SETNX + WAL Constraints
Cutoff Management	Batch window query evaluation	Event-time watermarking + state timers
Downstream Integration	REST APIs & Webhooks	Debezium CDC + Kafka Event Hub
Operational Governance	Built-in Maker-Checker Guard	Distributed RBAC + Policy-as-Code Engine

Vivriti Co-Lending Control Tower Architectural Documentation — Certified Production Ready.